PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Department of Computer Science and Engineering

P U B C S E F E S T - 2 6

HACKATHON 2026

P R O B L E M S E T

Blockchain and Cybersecurity · Artificial Intelligence and Machine Learning
Smart Campus Services · Academic and EdTech Platforms · Animated Short Film

DEVELOPMENT PERIOD	SUBMISSION DEADLINE	FINAL EVALUATION	TEAM SIZE
14 to 29 Oct 2026	**29 Oct 2026, 11:59 AM**	**30 Oct 2026**	**5 Students**

ORGANISED BY

Department of Computer Science and Engineering
Pundra University of Science & Technology, Bogura

Problem List

Each team selects one problem from the list below. Allocation is confirmed by the organising committee so that problems are spread across teams. Track E is a creative track in which teams produce an animated short film instead of software.

Track A: Blockchain and Cybersecurity	
A1	PUB CertChain: Blockchain-Based Academic Certificate Verification
A2	PUB Helping Hand: Blockchain-Based Transparent Fundraising and Student Assistance Platform
A3	PUB PhishGuard: Phishing Detection and Awareness Platform
A4	PUB MediVault: Secure Digital Health Record and Consent Management
Track B: Artificial Intelligence and Machine Learning	
B1	PUB NoticeLens: AI-Powered Bengali/English Notice and Document Intelligence
B2	PUB AssessAI: AI Question Preparation, Assessment and Learning Portal
B3	PUB SkillSync: AI-Powered Skill Matching, Productivity and Academic Early-Warning Platform
B4	PUB AttendEye: Computer-Vision-Based Smart Attendance System
Track C: Smart Campus Services	
C1	PUB CampusMate: Smart Campus Companion with Academic Planning, Course Registration and AI Helpdesk
C2	PUB SafeCampus: Emergency Response and Campus Safety Application
C3	SmartRent BD: Student Housing and Roommate Matching Platform
C4	PUB CampusRide: Campus Transport Scheduling and Operations Portal
C5	PUB FestHub: Campus Event, Competition and Coding Contest Operations Platform
Track D: Academic and EdTech Platforms	
D1	PUB SuperVise: Project and Thesis Supervision and Collaboration Portal
D2	PUB GradPath: Higher-Study Discovery and Application Workspace
D3	PUB RoboRoute: Campus Delivery Robot: A Programming and Logic Game
Track E: Animated Short Film	
E1	A Day in the Life of a CSE Student
E2	From Student to Software Engineer
E3	The Code That Changed Our Department

Creativity is part of the challenge. The requirements on each problem page define the minimum scope. Teams are free to add new features, modify the listed ones, or approach the problem in their own way, as long as the core requirements are met. Original, useful ideas are rewarded under the Innovation criterion.

Track A: Blockchain and Cybersecurity

A1. PUB CertChain

Blockchain-Based Academic Certificate Verification

Problem Description

Employers, foreign universities and scholarship boards usually verify a graduate's certificate by emailing the Registrar's office or by checking a scanned copy, which is slow and makes forged certificates hard to detect. Build a system in which the university issues digitally verifiable certificates and transcripts, records their fingerprints on a blockchain, and allows anyone to check a certificate within seconds without contacting the university.

Required Features

1. Secure login for university staff. Only issuer accounts registered in the smart contract can issue or revoke certificates.
2. Create student records and certificates (degree certificate, provisional certificate, transcript), each with a unique certificate ID, and generate the certificate as a PDF.
3. Compute a cryptographic hash (e.g. SHA-256) of the certificate and store the hash, certificate ID, issuer and issue date in a smart contract.
4. Print a QR code on every certificate that opens the verification page for that certificate.
5. Public verification page: verify by certificate ID, by scanning the QR code, or by uploading the PDF. The system recomputes the hash and compares it with the on-chain record.
6. Show a clear result: Valid, Invalid (modified or not found) or Revoked, together with the issuing authority and issue date.
7. Revoke a certificate with a recorded reason. A revoked certificate stays on the chain but always verifies as Revoked.
8. Batch issuance for a full graduating batch from a CSV file.
9. Student portal to view own certificates and share a time-limited verification link with an employer.
10. Audit trail of every issue, revoke and verification event, with the blockchain transaction hash.

Users and Roles

Controller of Examinations	Approves issuer accounts; co-signs revocations.
Registrar / Issuer	Creates, issues, batch-uploads and revokes certificates.
Student	Views own certificates and shares verification links.
Verifier (employer, university)	Verifies a certificate without creating an account.

Restrictions

- Do not store the certificate file or personal data (name, grades, contact) on the blockchain; store only hashes and references.
- Use a local or public test network (e.g. Hardhat, Ganache, Sepolia). No real cryptocurrency is required.
- The demonstration must show that changing even one character of a certificate makes verification fail.

Optional Features

Multi-signature issuance, re-issue with version history after correction, encrypted off-chain storage (e.g. IPFS), verification statistics, W3C Verifiable Credentials format.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track A: Blockchain and Cybersecurity

A2. PUB Helping Hand

Blockchain-Based Transparent Fundraising and Student Assistance Platform

Problem Description

When a student faces a medical emergency, cannot pay tuition, or is affected by a flood, help is usually collected informally through social media, and donors never see how the money was spent. Clubs raise funds for events in the same way. Build a platform where verified campaigns are created for students and university causes, every donation and expense is recorded on a blockchain, and spending rules are enforced by smart contracts instead of trust.

Required Features

1. Campaign request with category (medical, tuition, disaster relief, club or event, charity), target amount, deadline and supporting documents stored off-chain.
2. Verification workflow: a committee checks each request before it is published; verified campaigns carry a badge.
3. Emergency fund request with a shorter review window and higher listing priority.
4. Donations through simulated payment or test tokens. Every donation is recorded on the chain and the donor receives a receipt with the transaction hash.
5. Public campaign page showing target, amount raised, number of donors, current balance and the full transaction history.
6. Spending requests by the campaign owner with purpose, amount and evidence (bills, receipts).
7. The smart contract releases funds only after approval by at least k of n committee members, and blocks spending above the available balance and duplicate claims.
8. Campaign closing with a final report (raised, spent, remaining) and a defined rule for the remaining balance: refund to donors or transfer to a general welfare fund.
9. Donor dashboard showing donation history and how each donation was used.
10. Campaign priority ranking based on urgency, deadline and funding gap, and an analytics view for administrators.

Users and Roles

Campaign Owner / Beneficiary	Requests a campaign, submits spending requests with evidence.
Donor	Donates, tracks the use of donations, downloads receipts.
Verification Committee	Verifies campaigns and approves or rejects spending requests.
Administrator / Auditor	Manages users and categories; views all records and reports.

Restrictions

- No real money may be collected; use simulated payments or a test network.
- Personal, medical and financial details must not be stored on the chain. A beneficiary may choose to stay anonymous on the public page.
- Spending rules must be enforced in the smart contract, not only in the user interface.

Optional Features

Automatic refund when the target is not reached, milestone-based fund release, QR donation receipts, donor-selected spending purpose, matching funds from sponsors.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track A: Blockchain and Cybersecurity

A3. PUB PhishGuard

Phishing Detection and Awareness Platform

Problem Description

Students and staff regularly receive fake scholarship offers, mobile-banking reward messages, fake job letters and links to pages that imitate university or bank login portals. Build a system that analyses a suspicious URL, email or SMS, estimates how likely it is to be phishing, explains the reasons, and helps users learn to recognise phishing themselves.

Required Features

1. Accept three kinds of input: a URL, a full email (pasted with headers or uploaded as .eml), and SMS or chat text.
2. URL analysis: length, IP address as host, '@' symbol, number of subdomains, punycode and look-alike characters, suspicious top-level domains and URL shorteners.
3. Typosquatting check against a list of legitimate domains (e.g. the university, local banks and mobile financial services).
4. Email analysis: sender and reply-to mismatch, display-name spoofing, SPF/DKIM/DMARC results from the headers, urgency and credential-request wording, risky attachments (double extensions, macro files).
5. Keyword detection in both Bangla and English for SMS and message scams.
6. A machine-learning classifier trained on a public phishing dataset, combined with the rule-based indicators. Report accuracy, precision, recall and F1-score on a held-out test set.
7. Risk score from 0 to 100 mapped to Low, Medium or High.
8. An explanation that lists every indicator that contributed to the result.
9. Awareness module: a quiz built from synthetic phishing and genuine examples, with feedback after each answer and score history.
10. Admin dashboard with scan history, most frequent indicators, and allow-list / block-list management.

Users and Roles

User (student or staff)	Submits items for analysis, takes awareness quizzes.
Security Administrator	Manages lists and datasets, reviews scan history and statistics.

Restrictions

- Use only public, provided or synthetic datasets. Do not send phishing messages to real people.
- Do not collect real passwords or credentials and do not build copies of real login pages.
- Do not open live suspicious links on the demonstration machine; use stored page data or an isolated sandbox.

Optional Features

Browser extension, QR-code phishing detection, explainable AI (SHAP or LIME), email-client plug-in, a simulated phishing exercise for consenting team members only.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track A: Blockchain and Cybersecurity

A4. PUB MediVault

Secure Digital Health Record and Consent Management

Problem Description

Patients carry paper prescriptions and test reports from one doctor to another. Records are lost, and when they are shared through messaging apps there is no control over who sees them. Build a platform where patients own their health records, grant and withdraw access for doctors and hospitals, and every access is written to a tamper-evident audit trail.

Required Features

1. Registration for patients, doctors (with registration number checked by the administrator) and hospitals or diagnostic centres.
2. Upload and organise records by type: prescription, lab report, imaging, discharge summary, vaccination; each with date, provider and tags.
3. Encrypt records at rest; encryption keys must not be stored alongside the data.
4. Consent management: the patient grants a doctor or hospital access to selected record types for a chosen period and can withdraw access at any time.
5. Doctors see only the records they have consent for, and can add prescriptions and clinical notes to the patient's history.
6. Emergency (break-glass) access to a minimal profile (blood group, allergies, chronic conditions, current medication). A reason is mandatory and the patient is notified.
7. Record the hash of every record and every grant, withdrawal and access event on a blockchain ledger.
8. Integrity check that detects whether a stored record was changed after upload.
9. QR health card that shows the emergency profile and lets a doctor send an access request.
10. Patient timeline of the full medical history, plus an access log showing who viewed what and when.

Users and Roles

Patient	Uploads records, grants and withdraws consent, views the access log.
Doctor	Views consented records, adds prescriptions and notes, requests access.
Hospital / Diagnostic Centre	Uploads test reports and discharge summaries for a patient.
Administrator	Verifies doctors and institutions; cannot read record contents.

Restrictions

- Use synthetic patient data only. No medical data on the blockchain; only hashes and consent events.
- Access control must be enforced on the server and all traffic must use HTTPS.

Optional Features

HL7 FHIR data format, family or guardian access, medicine reminders, OCR for handwritten prescriptions, doctor appointment booking.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track B: Artificial Intelligence and Machine Learning

B1. PUB NoticeLens

AI-Powered Bengali/English Notice and Document Intelligence

Problem Description

University notices arrive as scanned PDFs, photos shared in Facebook groups and printouts on notice boards, in Bangla, English or a mix of both. Students miss deadlines because the required action is buried inside a long notice. Build a system that reads notices in either language, extracts the information a student must act on, and answers questions about them.

Required Features

1. Accept scanned PDFs, photos of printed notices, digital PDFs and plain text.
2. Image preprocessing (deskew, noise removal, cropping) and OCR for Bangla, English and mixed-language notices.
3. Extract structured fields: title, issuing office, date of issue, deadlines, event date and time, venue, target students (department, batch, semester), required action, fees and contact.
4. Convert Bangla dates and digits (for example ১৫ অক্টোবর ২০২৬) into standard dates.
5. Short summary of each notice in both Bangla and English.
6. Classify notices as exam, registration, fee, holiday, event, scholarship, result or general.
7. Searchable notice archive with filters by category, department and date.
8. Question answering over stored notices, citing the source notice and the matching passage.
9. Personalised feed: students choose their department and batch, see only relevant notices, get deadline reminders and can export deadlines to a calendar (.ics).
10. Evaluation on at least 30 notices: report OCR character error rate and extraction accuracy for each field.

Users and Roles

Student	Reads the personalised feed, searches and asks questions.
Office Staff / Publisher	Uploads notices, corrects extracted fields, publishes.
Administrator	Manages users, departments and categories.

Restrictions

- Use public university notices or synthetic notices.
- Extracted fields must be reviewed by staff before publication, and answers must come only from stored notices.

Optional Features

Handwritten notice support, text-to-speech reading, Messenger or Telegram bot, duplicate notice detection, highlighting changes between a notice and its revised version.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track B: Artificial Intelligence and Machine Learning

B2. PUB AssessAI

AI Question Preparation, Assessment and Learning Portal

Problem Description

Teachers spend many hours writing questions, assembling papers and marking written answers, while students receive little feedback between examinations. Build an AI-assisted portal in which teachers generate and curate questions from their own course materials, run practice and formal assessments, and evaluate written answers with teacher-approved criteria, while students receive explanations grounded in the course materials and targeted practice.

Required Features

1. Course setup: upload materials (slides, notes, PDFs) and organise them by topic and course learning outcome (CLO).
2. Generate MCQ, short and broad questions from the materials, each with difficulty, Bloom's level, CLO mapping, a suggested answer and a source reference.
3. Teachers edit, approve or reject generated questions and maintain a searchable question bank.
4. Assemble a question paper and answer key with configurable sections, marks and choices; show topic and CLO coverage; export to PDF or Word.
5. Practice sessions and timed formal assessments with MCQ and typed written answers, autosaved drafts and recorded submission time.
6. Automatic marking of MCQ, and AI evaluation of written answers against a teacher-approved marking scheme, giving marks per criterion with a short justification.
7. Teachers review, adjust and approve marks before they are published.
8. After an attempt, students receive explanations based on the course materials and follow-up practice on the topics they got wrong.
9. Progress tracking of attempts, scores and topic-wise performance for each student and for the class.
10. Assessment integrity for formal tests: question and option shuffling, time limits, tab-switch logging, and no AI tutor access during the test.

Users and Roles

Teacher	Uploads materials, curates questions, builds papers, moderates marks.
Student	Practises, takes assessments, reviews feedback and progress.
Administrator	Manages courses, users and enrolments.

Restrictions

- AI-generated marks are never published without teacher approval.
- Every generated question and explanation must refer to the course material it came from.

Optional Features

Bangla support, adaptive difficulty, spaced revision, alternative paper sets, similarity check on written answers, class analytics.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track B: Artificial Intelligence and Machine Learning

B3. PUB SkillSync

AI-Powered Skill Matching, Productivity and Academic Early-Warning Platform

Problem Description

Students struggle to find teammates with the right skills for projects and competitions, teams lose track of who is doing what, and teachers often notice a struggling student only after the final examination. Build a platform that forms teams by matching skills, helps students plan and track their work, and uses machine learning to flag students at academic risk early, with reasons an advisor can act on.

Required Features

Skill profile and team matching

1. Profile with skills and self-rated level, frameworks, interests, past projects, GitHub link and weekly availability, with visibility control for each field.
2. Project or competition posts with required skills and team size; search for members with filters.
3. Matching algorithm that ranks candidates by skill complementarity, shared interest and availability, and shows how the score was calculated.
4. Join requests, invitations and team management.

Planning and progress

5. Team task board with owners, priorities, deadlines and dependencies; warnings for overdue work and overloaded members.
6. Personal weekly planner with focus-session timer, comparing planned and actual effort.
7. AI suggestions for breaking a task into steps and revising a plan; the user accepts or edits them.

Academic early warning

8. Import academic data (attendance, quizzes, assignments, mid-term marks, previous GPA) from CSV.
9. Train and compare at least two models, predict Low, Medium or High risk, report accuracy, precision, recall and F1-score, and justify the final choice.
10. Explain each prediction with its main contributing factors (e.g. SHAP) and suggest interventions on an advisor dashboard.

Users and Roles

Student	Maintains profile, joins teams, plans tasks, sees own risk report.
Team Leader	Posts projects, accepts members, manages the task board.
Teacher / Advisor	Views risk reports for advisees and records interventions.
Administrator	Manages users and imports academic data.

Restrictions

- Use synthetic or anonymised academic data. A risk report is visible only to the student and the student's advisor.
- No automatic action may be taken against a student based on a prediction alone.

Optional Features

GitHub activity integration, team compatibility score, reflection journal, AI-generated progress feedback, peer evaluation within teams.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track B: Artificial Intelligence and Machine Learning

B4. PUB AttendEye

Computer-Vision-Based Smart Attendance System

Problem Description

Calling the roll wastes class time and paper sign-in sheets allow proxy attendance. Build a system that takes attendance from a live camera feed or a photo of the class, recognises enrolled students, lets the teacher confirm uncertain cases, and keeps reliable attendance records for every course.

Required Features

1. Student enrolment with consent, capturing several face samples per student and linking them to student ID, course and section.
2. The teacher starts a class session (course, section, time window) and captures attendance from a live camera feed or an uploaded classroom photo.
3. Detect and recognise multiple faces in a single frame and mark matched students present.
4. Confidence threshold: unknown or low-confidence faces are sent to the teacher for review instead of being marked automatically.
5. Liveness or anti-spoofing check against a printed photo or a face shown on a phone screen.
6. Manual correction by the teacher; every correction is logged with time and reason.
7. Students see their attendance percentage for each course and receive an alert when it falls below a configurable limit.
8. Course-wise and date-wise reports with CSV or PDF export.
9. Evaluation on a small test set under different lighting and angles: report precision and recall (or false-accept and false-reject rates) and justify the chosen threshold.

Users and Roles

Student	Enrols face samples with consent and views own attendance.
Teacher	Runs sessions, reviews flagged faces, corrects records, exports reports.
Administrator	Manages courses, sections, users and the attendance limit.

Restrictions

- Use only faces of consenting volunteers (for example, team members). Do not use scraped images of real students.
- Store face embeddings rather than raw photos where possible, and protect all biometric data.
- Pretrained detection and recognition models may be used, but the team must explain the complete pipeline.

Optional Features

Handling masks and partial occlusion, QR plus face two-factor check-in, CCTV or IP-camera stream, offline mobile mode, attendance analytics dashboard.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track C: Smart Campus Services

C1. PUB CampusMate

Smart Campus Companion with Academic Planning, Course Registration and AI Helpdesk

Problem Description

Class routines, exam schedules, notices and academic rules are spread across notice boards, social media groups and office visits. Course registration and routine preparation are done by hand, so clashes are found late. Build one platform where students follow their academic life, the department plans the semester and registers students without conflicts, and an AI helpdesk answers academic questions from official documents.

Required Features

Student companion

1. Secure login and a personal dashboard with class schedule, exam schedule, personal timetable and room locations.
2. Notices, event notifications and a faculty directory.

Academic planning and registration

3. Manage curricula, prerequisites, semester offerings, sections, rooms and teacher availability.
4. Course registration with prerequisite, credit-limit, seat-capacity and timetable-clash checks; advisor approval and add/drop requests.
5. Assign teachers to courses and calculate normal and extra teaching load using configurable rules.
6. Create and revise the routine; detect teacher, room, section and registered-student clashes before publishing; arrange make-up classes.
7. Publish approved schedules, notify affected users and keep the revision history.

AI helpdesk

8. Chat assistant that answers questions about courses, prerequisites, faculty, labs, notices and the academic calendar using retrieval over official department documents.
9. Every answer cites its source, Bangla and English questions are supported, and the assistant says so when the documents do not contain the answer.

Users and Roles

Student	Views schedules and notices, registers for courses, uses the helpdesk.
Teacher	Sets availability, views assigned courses and load, proposes make-up classes.
Academic Advisor	Approves registrations and add/drop requests.
Department Coordinator	Manages offerings, routine, notices and helpdesk documents.

Restrictions

- Any platform may be used: web, mobile or both. Authentication and role-based access must be enforced on the server.
- Demonstrate one department and one semester with sample student academic histories.

Optional Features

Campus map, push notifications, automatic timetable generation, clash-repair suggestions, waiting lists, calendar export.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track C: Smart Campus Services

C2. PUB SafeCampus

Emergency Response and Campus Safety Application

Problem Description

When a student faces harassment, an accident, a medical problem or a fire on campus, the quickest help is usually nearby security staff or the proctor's office, but students often do not know whom to contact and nothing is recorded. Build an application that lets anyone on campus raise an alert or report an incident in seconds, routes it to the right responders with location, and tracks it until it is resolved.

Required Features

1. SOS button (hold to confirm, with a short cancel countdown) that sends identity, live location and time to the security desk and to the user's personal emergency contacts.
2. Incident reports by category (medical, harassment, ragging, fire, accident, theft, hazard) with description, photo or video, and location as a map pin or building and room.
3. Anonymous reporting for sensitive categories such as harassment and ragging.
4. Responder dashboard showing active alerts on a map, ordered by severity, with assignment to a responder and acknowledgement.
5. Status tracking (Reported, Acknowledged, Responder on the way, Resolved) with timestamps visible to the reporter, and chat between reporter and responder.
6. Campus emergency directory (security, medical centre, proctor, fire service, ambulance) with one-tap calling.
7. Broadcast alerts from the security office to all users (for example fire drill or flood warning) with read confirmation.
8. Offline fallback: queue the report and send it when the connection returns, or prepare an SMS.
9. Incident history and reports: response time, incidents by category and location, and an incident heat map.

Users and Roles

Student / Staff	Raises SOS alerts, reports incidents, follows status.
Responder	Security staff, medical officer or proctor; accepts and resolves incidents.
Security Administrator	Manages responders, broadcasts alerts, views reports.

Restrictions

- Label the application as a prototype and provide a clearly marked test mode.
- Never contact real emergency services (such as 999) during testing.
- Location is shared only with permission and only while an alert is active. Anonymous reports must not reveal the reporter.

Optional Features

Nearest-responder selection, fake-call feature, SMS simulation, walk-with-me live location sharing, shake-to-alert.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track C: Smart Campus Services

C3. SmartRent BD

Student Housing and Roommate Matching Platform

Problem Description

Students who move away from home for university struggle to find safe and affordable rooms, hostel seats or mess seats near campus. They depend on Facebook groups, brokers and word of mouth, and fake listings that ask for advance payment are common. Build a platform that lists verified accommodation near the university, matches compatible roommates, and helps roommates manage shared expenses.

Required Features

1. Owner listings for rooms, flats, mess seats and hostel seats with rent, deposit, utilities, facilities (Wi-Fi, generator, water, meals), gender policy, photos, map location and availability date.
2. Search and filter by distance from campus, budget, type, facilities and gender policy, with a map view.
3. Listing verification: the administrator checks owner identity and marks inspected listings as Verified.
4. Roommate profile covering budget, preferred area, sleep schedule, study habits, cleanliness, smoking, food and guests.
5. Compatibility matching with a score and explanation; contact details are shared only after both sides agree.
6. Visit and booking requests: the student requests a visit, the owner schedules it, and booking is confirmed.
7. Ratings and reviews of listings and owners by verified tenants only.
8. Report fake listings or advance-payment scams; reported listings are hidden until reviewed.
9. Shared expense manager for roommates: rent split, utility bills, mess meal accounts, balances and a monthly summary.
10. Maintenance requests from tenant to owner with status, and alerts for new listings that match saved searches.

Users and Roles

Student / Tenant	Searches, finds roommates, books, reviews, shares expenses.
Owner / Landlord	Posts listings, schedules visits, handles maintenance requests.
Administrator / Moderator	Verifies owners and listings, handles reports.

Restrictions

- No real payments; use simulated records. Personal profiles must have visibility controls.

Optional Features

Waiting list for popular listings, rent agreement generation, in-app chat, fair-rent estimation, mess manager rotation.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track C: Smart Campus Services

C4. PUB CampusRide

Campus Transport Scheduling and Operations Portal

Problem Description

University buses run on fixed routes, but schedules are shared on paper or by phone, students do not know when a trip is delayed or cancelled, and the transport office cannot see whether a bus is overloaded. Build a portal that manages routes, buses, drivers and trips, keeps students informed, and gives the transport office reliable data on delays, demand and vehicle usage.

Required Features

1. Manage routes with ordered stops, service days, travel times and pickup instructions.
2. Maintain bus capacity, vehicle status (available, under maintenance, temporarily unavailable) and driver availability.
3. Schedule recurring and one-off trips, and detect overlapping vehicle or driver assignments.
4. Students view schedules, save preferred routes and record travel intentions; compare expected demand with bus capacity.
5. Driver or dispatcher interface to record departure, stop arrivals, delays, completion and cancellation.
6. Handle breakdowns and driver absence by finding an available replacement and flagging capacity shortages.
7. Publish trip updates and notify affected passengers about delays, cancellations or changed arrangements.
8. Record maintenance needs, incidents and passenger feedback, and track follow-up actions to resolution.
9. Trip history and reports covering delays, cancellations, passenger demand and vehicle usage.

Users and Roles

Student / Passenger	Views schedules, saves routes, records travel intention, gives feedback.
Driver	Records trip progress, delays and incidents.
Transport Coordinator	Manages routes, buses, drivers and trips; handles disruptions.

Restrictions

- Demonstrate one campus with a small set of routes, buses and drivers. Manually reported trip status is enough for the core workflow.

Optional Features

Live GPS tracking, arrival-time estimation, seat reservation, boarding check-in, route optimisation.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track C: Smart Campus Services

C5. PUB FestHub

Campus Event, Competition and Coding Contest Operations Platform

Problem Description

Department fests run seminars, competitions and programming contests with dozens of volunteers, and most of the coordination happens in chat groups and spreadsheets. Build a platform that organises an event from planning to results: registration, scheduling, volunteers, judging, and a built-in programming contest with automatic judging.

Required Features

Events and competitions

1. Create an event with activities (seminar, workshop, competition), eligibility, registration deadline, participant limit and venue.
2. Individual and team registration with invitations, approval, cancellation and waiting list.
3. Schedule activities and rounds; flag venue clashes and overlapping volunteer duties.
4. Judging for general competitions: versioned entries, judge assignment, rubric scores and organiser-approved results.

Volunteers and operations

5. Volunteer availability, duty and shift assignment, and coordinator-approved replacement requests.
6. QR check-in for participants, volunteer attendance and task completion records.
7. Announcements, issue tracking with owner and resolution, and a readiness view (uncovered shifts, missing judges).

Programming contest

8. Problem bank with statements, constraints, samples and hidden test cases; contests with start and end time and ICPC-style scoring.
9. Submissions in at least one language judged in an isolated sandbox with time and memory limits, verdicts (AC, WA, TLE, RE, CE) and a live scoreboard with freeze.
10. Practice mode after the contest, editorials released after the end, and topic-wise progress for each student.

Users and Roles

Organiser / Coordinator	Creates events, schedules, approves results, manages volunteers.
Participant / Team Leader	Registers, submits entries or code, views results.
Volunteer	Takes shifts, checks in participants, reports issues.
Judge / Problem Setter	Scores entries; sets contest problems and test data.

Restrictions

- Submitted code must never run directly on the server; use a container or an existing judge engine (e.g. Judge0). The contest logic must be built by the team.

Optional Features

Certificates, sponsor tracking, equipment allocation, plagiarism check for code, AI feedback on submissions.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track D: Academic and EdTech Platforms

D1. PUB SuperVise

Project and Thesis Supervision and Collaboration Portal

Problem Description

Final-year projects and theses are managed through email, chat groups and printed forms. Coordinators cannot see which projects are behind schedule, feedback gets lost, and earlier versions of a report are overwritten. Build a portal that manages projects, theses and research work from proposal to final evaluation.

Required Features

1. Proposal submission with title, abstract, research area, team members and supervisor preference.
2. The coordinator assigns supervisors while respecting a maximum load per supervisor.
3. Supervisors approve, reject or request revision of a proposal; the approved scope is saved as a version.
4. Milestones (literature review, design, implementation, draft report, final report) with deadlines and member responsibilities; overdue work is flagged.
5. Progress reports with evidence, blockers and next steps, and versioned deliverables with the full submission history.
6. Supervisor feedback, comments and revision requests on each submission, with approval status.
7. Private project chat for team members and supervisor, with saved messages and unread indicators.
8. Meeting log recording date, attendees, decisions and follow-up actions with owner and deadline.
9. Coordinator dashboard with status of all projects, supervisor loads and projects at risk.
10. Final submission checklist (report, code, poster, similarity report), defence scheduling and rubric-based evaluation by the board.

Users and Roles

Student / Researcher	Submits proposals, reports and deliverables; chats with supervisor.
Supervisor	Reviews work, gives feedback, approves milestones.
Coordinator	Assigns supervisors, monitors progress, schedules defences.
Board Member / Examiner	Evaluates the defence with the rubric.

Restrictions

- Students can see only their own project. Earlier submissions are never overwritten.

Optional Features

Similarity checking, AI progress summaries, deadline reminders, report format checking, searchable archive of past theses.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track D: Academic and EdTech Platforms

D2. PUB GradPath

Higher-Study Discovery and Application Workspace

Problem Description

Students who plan higher study abroad collect information about programmes, professors, scholarships and deadlines from many websites and keep it in scattered notes. Requirements are missed, recommendation letters are requested late, and costs are underestimated. Build a workspace that helps a student find suitable opportunities, prepare applications, plan funding and track every outcome.

Required Features

1. Student profile with academic background, research interests, skills, publications, experience and supporting documents.
2. Searchable records of programmes, universities, professors, labs and funding, each with its source and the date it was last checked.
3. Profile-to-opportunity matching that explains the fit and lists requirement gaps.
4. Compare shortlisted opportunities by research fit, requirements, funding, estimated cost and deadline; keep notes and mentor feedback.
5. Eligibility check against recorded requirements; track required tests (IELTS, GRE), scores and planned test dates.
6. Application-specific checklists and tasks with deadlines, dependencies and reminders.
7. Document manager for SOP, CV and research statements with versions, mentor comments and review status.
8. Track recommendation requests, professor outreach emails, pending replies and follow-ups.
9. Scholarship and funding tracker, and a cost estimate (fees, living cost, travel) using entered amounts, currencies and stated assumptions.
10. Application readiness view, plus records of submissions, interviews, decisions and offer-response deadlines.

Users and Roles

Student / Applicant	Builds profile, shortlists, prepares and tracks applications.
Mentor	Reviews documents, gives feedback on choices and readiness.
Administrator	Maintains opportunity records and user accounts.

Restrictions

- Opportunity data may be entered manually or imported from a sample dataset; do not scrape websites against their terms of use.

Optional Features

AI-assisted matching, SOP feedback, interview practice, automatic deadline updates, calendar integration.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track D: Academic and EdTech Platforms

D3. PUB RoboRoute

Campus Delivery Robot: A Programming and Logic Game

Problem Description

New programmers often find loops, conditions and functions abstract. Build an educational game in which students program a simulated robot to collect and deliver parcels on a campus map, learning sequencing, loops, conditions and functions by watching their program run step by step.

Required Features

1. A campus grid with paths, buildings, pickup points, delivery destinations and obstacles.
2. Build and edit programs with command blocks or a simple text language, supporting move, turn, pick up, deliver, loops, conditions (e.g. path ahead clear, parcel here) and reusable functions.
3. At least five missions of increasing difficulty with clear objectives and completion rules, where later missions require loops, conditions and functions.
4. Run, pause, step and restart, highlighting the current instruction and showing robot position, direction and parcel status.
5. Clear messages for blocked movement, invalid actions and incomplete deliveries, with contextual hints and the option to revise and retry.
6. Score each attempt on delivery success, number of steps and program length, and award stars.
7. Save programs, attempt history and mission progress for each player.
8. Mission editor for instructors to design maps, place parcels and obstacles, and set objectives.
9. Instructor view of class progress: missions completed, attempts and common errors.

Users and Roles

Student / Player	Writes and runs programs, completes missions, sees progress.
Instructor	Creates and edits missions, reviews class progress.

Restrictions

- A 2D simulation with a limited command set is sufficient; physical robot hardware is optional.

Optional Features

Battery or time limits, multiple parcels at once, sharing challenges with friends, AI debugging hints, leaderboard.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Track E: Animated Short Film

Animated Short Film: General Guidelines

Overview

Each team creates a 7 to 10 minute animated short film set in the university or the CSE department, based on one of the three themes E1, E2 or E3. The team writes the story and dialogue itself. The finished film must show real animation work, storytelling, editing, sound design and creative direction. A sequence of still images or a single generated video does not meet the requirements.

Production Requirements

1. Length between 7 and 10 minutes including titles and credits.
2. Original story, script and dialogue written by the team, in Bangla or English. Subtitles are required if the dialogue is in Bangla.
3. Character sheets and environment designs, with characters that keep a consistent appearance throughout.
4. A storyboard covering every major shot, and an animatic (a rough timed version of the storyboard) made before final animation.
5. Actual animation of characters, camera and environment: movement, expressions and transitions, not only panning over still images.
6. Voice recorded by team members wherever possible, plus ambience, sound effects and music that do not overpower the dialogue.
7. Edited and composited final film with titles and credits naming every team member and any external assets used.
8. Final export as MP4 (H.264), 1920 × 1080, at least 24 frames per second.

Recommended Production Pipeline

Stage	What the team does
1. Story and script	Write the story, dialogue, narration and scene descriptions.
2. Design	Create characters, props, locations and a colour and style reference.
3. Storyboard	Draw the major shots. Stick figures are acceptable; composition and camera direction matter.
4. Animatic	Turn the storyboard into a rough timed video with temporary dialogue and music.
5. Asset creation	Build reusable characters, backgrounds, props, screens and effects.
6. Animation	Animate characters, camera movement, expressions and environment.
7. Voice and sound	Record dialogue; add ambience, Foley, effects and music.
8. Editing and compositing	Combine animation, sound, titles and effects; check timing and continuity.
9. Final review	Check story clarity, audio levels, spelling, credits and export quality.

Suggested Tools

Task	Free option	Alternative	Skill learned
2D animation	Krita, OpenToonz	Adobe Animate	Keyframes, timing, rigging
3D animation	Blender	Maya	Modelling, lighting, camera
Storyboard	Krita, Storyboarder	Photoshop	Shot planning
Video editing	DaVinci Resolve	Premiere Pro	Pacing, titles, audio
Compositing	Blender, DaVinci Resolve	After Effects	Layers, effects, colour
Audio	Audacity	Adobe Audition	Recording, cleanup, mixing
Vector assets	Inkscape	Illustrator	Reusable 2D assets

Recommended Story Structure (7 to 10 minutes)

Time	Story function
0:00 to 1:00	Hook and introduction: establish the character, problem or mystery.
1:00 to 3:00	Setup: introduce the university or CSE environment and the main characters.
3:00 to 6:00	Development: conflict, challenge, discovery or project.
6:00 to 8:30	Climax: the main problem is solved or the central realisation happens.
8:30 to 10:00	Resolution: emotional conclusion, message and credits.

Suggested Team Roles (5 members)

Role	Responsibility
Director / Scriptwriter	Story, script, scene planning and creative consistency.
Character and Environment Artist	Characters, backgrounds, props and visual assets.
Animator	Character animation, camera movement, transitions and effects.
Voice, Sound and Music Lead	Dialogue recording, Foley, ambience, music and audio cleanup.
Editor / Compositor	Animatic, final editing, compositing, titles and export.

Members may help one another; the role split is a starting point, not a restriction.

Quality Checklist

- A clear beginning, middle and end.
- Characters look and behave consistently.
- Shots have intentional framing and camera movement.
- Animation timing is believable.
- Dialogue is reasonably synchronised with mouth and body movement.
- Backgrounds and props establish the university or CSE setting.
- Sound effects and ambience support the visuals, and music does not overpower dialogue.
- Transitions are purposeful.
- Credits name every team member and any external assets.

Use of AI Tools

AI tools may help with brainstorming, rough visual references, temporary narration for the animatic, noise removal, subtitles, and repetitive production tasks. The script, storyboard, shot composition, character acting, animation decisions, editing and final creative direction must remain the team's own work. Evaluators will review the storyboard, animatic and project files along with the final film.

Track E: Animated Short Film

E1. A Day in the Life of a CSE Student

Concept

Follow one student from the first day in the CSE department through classes, labs, assignments, friendship, failure and growth. The film should feel honest and familiar to any CSE student, showing both the pressure and the reward of studying computer science.

Required Story Elements

1. First day and orientation in the department.
2. First programming class and the first program that does not compile.
3. A lab session and a late-night assignment with debugging.
4. Friendship and teamwork on a project or competition.
5. A clear setback (a failed exam, a rejected project, a lost contest) followed by growth.
6. A reflective ending, for example graduation or a senior advising a new student.

Theme-Specific Requirements

- The main character must change by the end of the film; show the change through actions, not only narration.
- Use at least three recognisable campus locations (classroom, lab, corridor, canteen or similar).

General Requirements

All production requirements in the Track E General Guidelines apply: 7 to 10 minutes, original script, storyboard and animatic, real animation, team-recorded voice where possible, credits, and 1080p MP4 export.

Intended Use

The best film may be used by the department for: orientation programmes, CSE Fest, department anniversaries and admission activities.

Be creative: the story elements above are a guide. Teams may reorder them, add characters and subplots, choose any visual style (2D, 3D, cut-out, stop-motion or mixed), and give the story their own voice, as long as the theme stays recognisable.

Track E: Animated Short Film

E2. From Student to Software Engineer

Concept

A career story that follows a student from choosing CSE, through struggling with programming and discovering different fields, to completing projects and entering professional life. The film should inspire new students and show that the path is built step by step.

Required Story Elements

1. Why the student chose CSE.
2. Early struggles with programming.
3. Discovering fields such as AI/ML, software engineering, cybersecurity and networking.
4. A team project that becomes a turning point.
5. Internship or career preparation (CV, interview, first job offer).
6. Graduation and future goals.

Theme-Specific Requirements

- Show at least two fields of computing visually, for example with animated screens, diagrams or metaphors.
- The film should carry one clear message for a first-year student, stated or shown by the end.

General Requirements

All production requirements in the Track E General Guidelines apply: 7 to 10 minutes, original script, storyboard and animatic, real animation, team-recorded voice where possible, credits, and 1080p MP4 export.

Intended Use

The best film may be used by the department for: an inspirational film and a promotional video for the department.

Be creative: the story elements above are a guide. Teams may reorder them, add characters and subplots, choose any visual style (2D, 3D, cut-out, stop-motion or mixed), and give the story their own voice, as long as the theme stays recognisable.

Track E: Animated Short Film

E3. The Code That Changed Our Department

Concept

A fictional mystery set inside the CSE department. Late one night, students discover an old computer holding an unfinished project left by earlier students. Following clues in the code, they uncover its purpose and complete it, connecting past, present and future generations of the department.

Required Story Elements

1. A late-night scene in a CSE lab.
2. Discovery of an old computer or an unfinished project.
3. A mystery around the code: what it does and who wrote it.
4. Investigation and teamwork, with clues found in comments, commits or old files.
5. References to the department's real labs, clubs, research or achievements.
6. Completion of the project and an emotional link between past and present students.

Theme-Specific Requirements

- Plant at least two clues early in the film that pay off at the end.
- Code shown on screen should look realistic; avoid random symbols.

General Requirements

All production requirements in the Track E General Guidelines apply: 7 to 10 minutes, original script, storyboard and animatic, real animation, team-recorded voice where possible, credits, and 1080p MP4 export.

Intended Use

The best film may be used by the department for: introducing the department to visitors and new students without feeling like a brochure.

Be creative: the story elements above are a guide. Teams may reorder them, add characters and subplots, choose any visual style (2D, 3D, cut-out, stop-motion or mixed), and give the story their own voice, as long as the theme stays recognisable.

Rules, Evaluation and Submission

Timeline

Problem set release and team registration	14 October 2026
Feature development	14 October 2026 to 29 October 2026, 11:59 AM
Submission deadline and feature freeze (GitHub repository link)	29 October 2026, 11:59 AM
Primary evaluation (repository, README and demo video)	29 October 2026, after the deadline
Bug fixes and UI improvements only	From the deadline until the final evaluation
Final on-site evaluation	30 October 2026

General Rules

1. Each team must have exactly 5 students. A student may be a member of only one team.
2. Each team works on one problem from this problem set, subject to allocation by the organising committee.
3. Teams may use any programming language, framework, database, cloud service or blockchain platform, and any web, mobile or desktop platform.
4. Open-source libraries and frameworks are allowed. Simply combining existing tools without meaningful implementation of your own will receive limited marks.
5. Teams may use any tools, including AI assistants, but every member must be able to explain the design, code and decisions of the project during evaluation.
6. Copying an existing project and presenting it as the team's own work leads to disqualification.
7. External people must not develop any part of the solution for a team.
8. No real money may be collected and no real personal, medical or financial data may be used. Use simulated or synthetic data.
9. The GitHub repository link must be submitted by 11:59 AM on 29 October 2026, with the demo video link included in its README. There is no late submission.
10. Feature freeze: after the deadline, teams may only fix bugs and improve the user interface. No new features may be added, and any feature added after the deadline is not marked.
11. The decision of the evaluation team is final.

Cybersecurity Safety Notice

All security testing and attack simulation must be performed only on systems, applications, datasets and networks owned by the team or explicitly authorised by the organisers. Participants must not scan, exploit, attack, disrupt or attempt unauthorised access to any external system, website, server, network or account. Any violation leads to immediate disqualification.

Project Evaluation

There is no formal presentation and no slides are required. Evaluation takes place in two stages.

- **Primary evaluation (29 October, after the deadline):** the evaluation team reviews each submitted repository, README and demo video, and records the features present at the deadline.
- **Final evaluation (30 October):** the evaluation team visits and checks every project in person, about 20 to 30 minutes per project. All five members must be present at their station and take part.

Tracks A to D

1. The team gives a short overview of the problem, the solution and the system design (about 3 minutes).
2. The team demonstrates every required feature live, role by role, using prepared sample data.
3. Everything shown will be tested. Evaluators will use the system themselves, enter invalid data, try to reach pages or data without permission, and check edge cases.
4. Any evaluator may question any member about any part of the project, and may ask to see the source code, database, smart contract or model training code.
5. A feature that does not work during evaluation is counted as not implemented. The demo video in the README is for reference and does not replace the live check.
6. The version at the deadline is the reference. Commits after the deadline may only fix bugs or improve the UI, and each such commit message must start with "fix:" or "ui:". Evaluators compare the final version with the deadline version; new features added later are not marked.
7. Teams set up their own laptops, phones and other devices before their evaluation slot. Internet access may be unreliable, so keep a local setup ready.

Track E (Animated Short Film)

1. The full film is screened.
2. The team shows its script, storyboard, animatic and project files open in the animation software, and explains how the film was made.
3. Evaluators may ask any member about their role and how specific scenes were produced.
4. After the deadline only corrections (audio, colour, subtitles, timing) are allowed; no new scenes may be added.

Submission Requirements: Tracks A to D

Only a GitHub repository with a complete README, including the demo video link, is required. No report or slides are needed.

- Git repository (GitHub or GitLab) with the complete source code. If the repository is private, give access to the evaluators. The commit history should show work across the development period. Do not commit passwords, API keys or .env files.
- README.md in the root of the repository containing:
  - Problem ID and title, team name, and members with name, student ID and role.
  - Demo video (mandatory): a link at the top of the README to a 5 to 10 minute screen recording (YouTube unlisted or Google Drive, viewable by anyone with the link) showing the main features. A repository without the video link in its README is treated as an incomplete submission.
  - A short description of the solution and the technology stack.
  - Feature status table: each required feature marked Done, Partial or Not done, followed by the team's additional features.
  - Step-by-step setup and run instructions, an example environment file, and sample login accounts for every role.
  - Screenshots of the main screens, and an architecture or database diagram where applicable.
  - Deployed link or installable build (for example an Android APK), if available.
  - Known limitations and team contribution (who did what).

Submission Requirements: Track E

- Final film as MP4 (H.264), 1920 × 1080, 7 to 10 minutes, shared through a Google Drive link.
- A folder with the script, character sheets, storyboard, animatic and editable project files.
- A README listing team members and roles, tools used, and any external assets with their source or licence.

Evaluation Criteria: Tracks A to D

Criterion	Marks	What evaluators look for
Problem Understanding	10	Clear grasp of the problem, users and requirements.
Functionality	25	Required features work correctly when tested by the evaluators.
Technical Implementation	20	Sound architecture, code quality and appropriate use of technology.
Security and Reliability	10	Authentication, access control, data protection and handling of invalid input.
Innovation	15	Original ideas, useful additions and creative modifications beyond the minimum scope.
UI/UX	10	Ease of use, clarity and accessibility of the interface.
Explanation and Q&A	10	Every member explains the work clearly and answers questions correctly.
Total	**100**	

Evaluation Criteria: Track E (Animated Short Film)

Criterion	Marks	What evaluators look for
Story and Script	25	Originality, structure, emotional impact and clarity.
Animation and Visual Quality	25	Movement, composition, consistency and visual design.
Sound and Editing	15	Dialogue, Foley, music, pacing and mixing.
Technical Execution	15	Production pipeline, storyboard and animatic, project files, export quality.
Creativity and CSE Integration	10	Effective and original use of the university and CSE setting.
Explanation of Process	10	Every member explains their role and how the film was made.
Total	**100**	

Final Note to Participants

The aim of this hackathon is not only to build a working application or film, but to show problem-solving ability, technical understanding, teamwork, creativity and responsible engineering. Keep your scope realistic, finish the core requirements first, and then use the remaining time to add the ideas that make your work stand out. A polished, working prototype is worth more than an unfinished system with many features.